Linear#

https://pytorch.org/docs/stable/generated/torch.nn.Linear.html

Description:

Applies an affine linear transformation to the incoming data: y=xAT+by = xA^T + by=xAT+b. This module supports TensorFloat32. On certain ROCm devices, when using float16 inputs this module will use different precision for backward. in_features (int) – size of each input sample out_features (int) – size of each output sample

Function Signature

```
class torch.nn.Linear(in_features, out_features, bias=True,
device=None, dtype=None)[source]#

Parameters

Shape:

Variables

extra_repr()[source]#
```

Returns:

Tensor

Code Examples

```
>>> m = nn.Linear(20, 30)
>>> input = torch.randn(128, 20)
>>> output = m(input)
>>> print(output.size())
torch.Size([128, 30])
```

Detailed Documentation

Documentation

Applies an affine linear transformation to the incoming data: y=xAT+by = xA^T + by=xAT+b.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

in_features (int) – size of each input sample

out_features (int) – size of each output sample bias (bool) – If set to False, the layer will not learn an additive bias. Default: True

Input: (∗,Hin)(*, H_\text{in})(∗,Hin) where ∗*∗ means any number of dimensions including none and Hin=in_featuresH_\text{in} = \text{in_features}Hin=in_features.

Output: (∗,Hout)(*, H_\text{out})(∗,Hout) where all but the last dimension are the same shape as the input and Hout=out_featuresH_\text{out} = \text{out_features}Hout =out_features.

weight (torch.Tensor) – the learnable weights of the module of shape (out_features,in_features)(\text{out_features}, \text{in_features}) (out_features,in_features). The values are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k), where k=1in_featuresk = \frac{1}{\text{in_features}}k=in_features1

bias – the learnable bias of the module of shape (out_features)(\text{out_features}) (out_features). If bias is True, the values are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=1in_featuresk = \frac{1}{\text{in_features}}k=in_features1

Return the extra representation of the module.

Runs the forward pass.

Resets parameters based on their initialization used in __init__.